

Metody Programowania Równoległego

Sprawozdanie

The Message Passing Interface (MPI)

Robert Mesek

Spis treści

1	Wstęp i opis środowiska	2
2	Część 1: Opóźnienia i charakterystyki przepustowości połączeń w klastrze	2
2.1	Metodologia i konfiguracja	2
2.2	Analiza wyników	2
3	Część 2: Przybliżanie liczby Pi metodą Monte Carlo	4
3.1	Opis algorytmu i implementacji	4
3.2	Przyspieszenie	4
3.3	Efektywność	5
3.4	Udział części sekwencyjnej	5
3.5	Czasy wykonania	6
3.6	Koszt komunikacji	7
4	Podsumowanie	8

1 Wstęp i opis środowiska

Celem niniejszego sprawozdania jest analiza wydajności komunikacji punkt-punkt oraz badanie skalowalności równoległego algorytmu przybliżania liczby π metodą Monte Carlo przy użyciu biblioteki MPI.

Do realizacji pomiarów wykorzystano dwa środowiska:

- **vCluster**: wykorzystany do testów komunikacji w klastrze (Część 1). Sprawdzono zarówno komunikację wewnątrz jednego węzła, jak i między różnymi węzłami.
- **Superkomputer Ares**: wykorzystany do testów skalowalności algorytmu Monte Carlo (Część 2). Testy przeprowadzono dla różnych konfiguracji liczby procesorów w obrębie jednego węzła.

2 Część 1: Opóźnienia i charakterystyki przepustowości połączeń w klastrze

2.1 Metodologia i konfiguracja

Pomiary wykonano metodą "ping-pong", mierząc czas przesłania komunikatu tam i z powrotem N razy (gdzie $N = 1001$), a następnie obliczając średnią. Przetestowano dwa tryby: `MPI_Send` oraz `MPI_Ssend` dla komunikatów o rozmiarach od 1 B do około 32 MB.

Wykorzystano dwie konfiguracje sprzętowe:

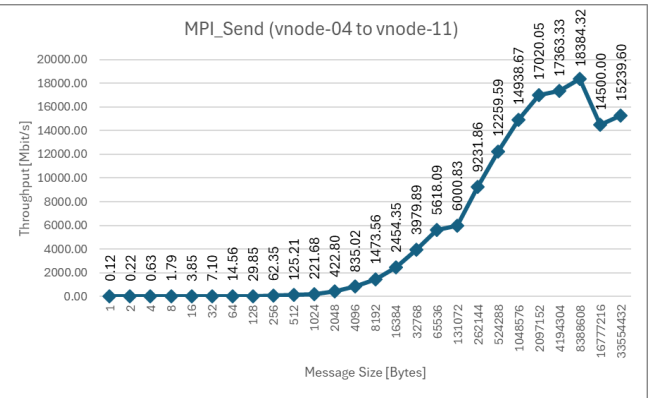
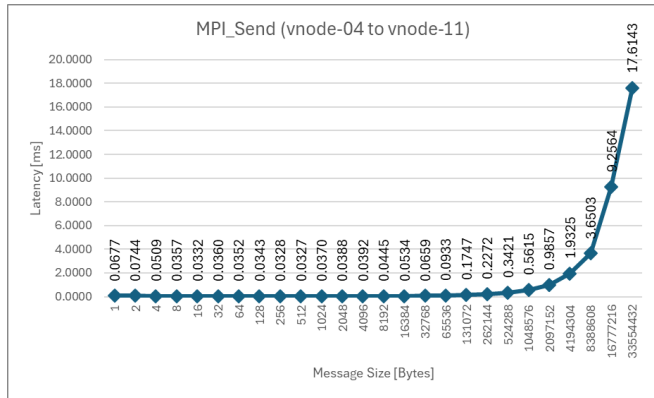
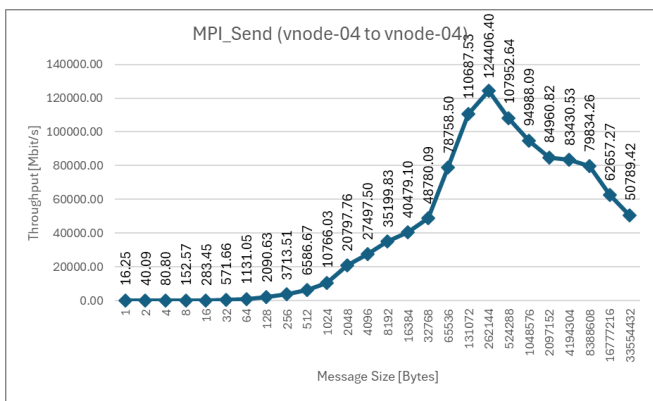
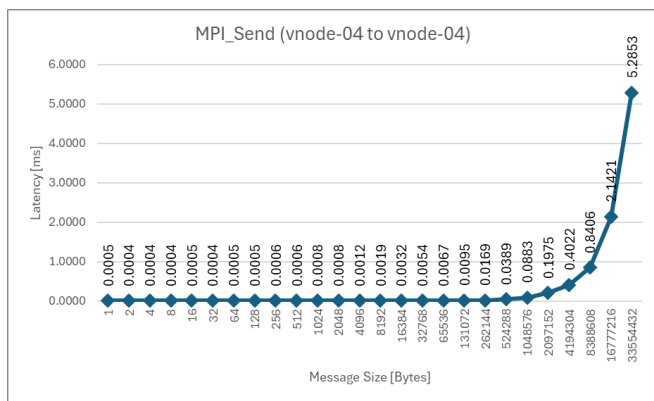
- Komunikacja wewnątrz jednego węzła (użycie dwóch procesorów na tym samym nodzie).
- Komunikacja między dwoma różnymi węzłami.

2.2 Analiza wyników

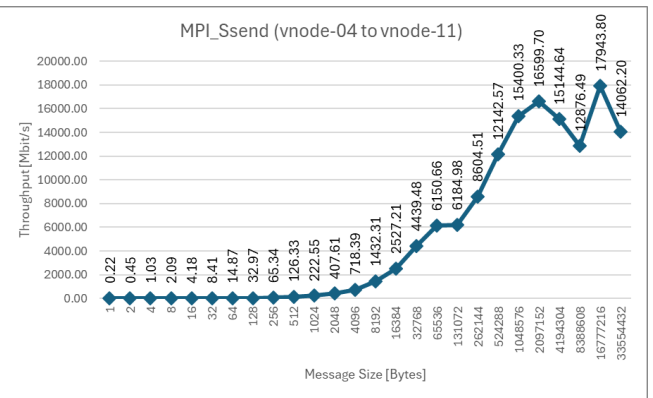
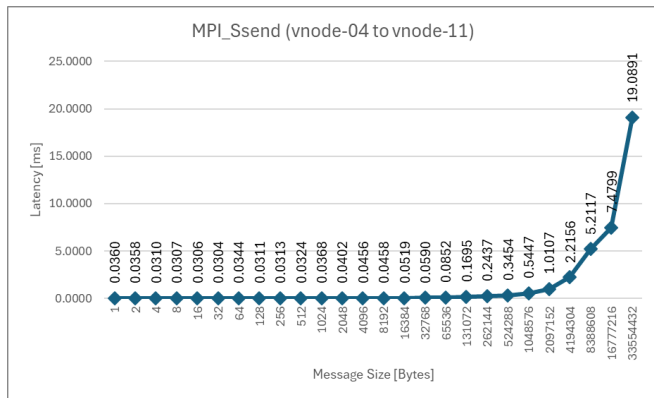
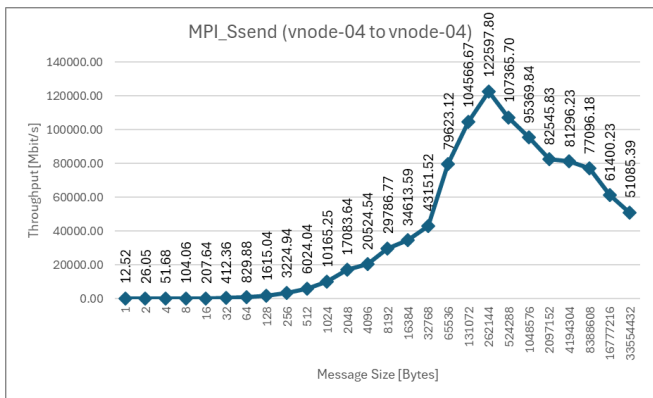
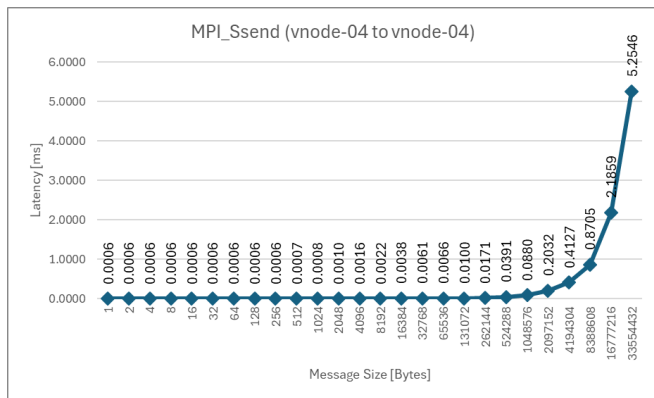
Najważniejsze obserwacje z przeprowadzonych pomiarów (Rysunki 1 i 2) są następujące:

- Dla komunikacji wewnątrz jednego węzła, opóźnienie jest znacznie niższe niż dla komunikacji między węzłami, co jest zgodne z oczekiwaniami ze względu na brak konieczności przechodzenia przez sieć.
- Dla małych komunikatów do około 1 KB opóźnienie utrzymuje się na identycznym poziomie, ale przepustowość dla tak małych komunikatów jest niska.
- Dla większych komunikatów przepustowość szybko rośnie, osiągając szczytowe wartości dla komunikatów w rozmiarze kilku MB, a następnie opada.

Obserwacje te są zgodne z intuicyjnymi oczekiwaniami i pokazują, jak istotne jest dobranie odpowiedniego rozmiaru komunikatów w zależności od charakterystyki sieci i wymagań aplikacji. Biblioteka MPI efektywnie zarządza komunikacją dopasowując się do aktualnych warunków uruchomienia, co widać w przypadku opóźnień w granicach 0.0005 ms dla połączeń wewnątrz węzła.



Rysunek 1: Wyniki benchmarku dla MPI_Send (układ 2x2).



Rysunek 2: Wyniki benchmarku dla MPI_Ssend (układ 2x2).

3 Część 2: Przybliżanie liczby Pi metodą Monte Carlo

3.1 Opis algorytmu i implementacji

Algorytm wykorzystuje stosunek pola koła do pola kwadratu: $\pi = 4 \cdot \frac{P_c}{P_s}$. Zrównoleglenie polega na podziale całkowitej liczby punktów do wylosowania pomiędzy procesy. Każdy proces liczy punkty wewnątrz okręgu, a wynik końcowy jest zbierany za pomocą operacji `MPI_Reduce`.

Wykonano testy dla dwóch wariantów skalowania:

- **Prawo Amdahla (Skalowanie silne):** liczba punktów jest stała, a liczba procesorów rośnie. W naszym przypadku wykorzystano równomierny podział pracy między procesory.
- **Prawo Gustafsona (Skalowanie słabe):** liczba punktów rośnie proporcjonalnie do liczby procesorów. Tutaj każdy procesor otrzymuje stały rozmiar problemu, więc im więcej procesorów, tym większy jest całkowity rozmiar problemu.

Prawo Amdahla mówi, że przyspieszenie jest ograniczone przez udział części sekwencyjnej, natomiast prawo Gustafsona zakłada, że część sekwencyjna pozostaje stała, natomiast zrównoleglona część rośnie wraz z liczbą procesorów. Przeanalizowano czasy wykonania w zależności od liczby procesorów, aby znaleźć przyspieszenie, efektywność oraz udział części sekwencyjnej dla obu wariantów skalowania.

Spodziewaliśmy się, dobrego zrównoleglenia, z powodu, że losowanie metodą Monte Carlo jest problemem naturalnie równoległym, czyli takim w którym część niemożliwa do zrównoleglenia jest pomijalna i nie ma wpływu na czas wykonania. Nie skupiamy się na dokładności przybliżenia liczby π , dopóki jest ona w granicach rozsądku, ponieważ nie jest to główny cel tego zadania. Sprawdzono trzy rozmiary problemu: mały (najniższy mierzalny czas), średni i duży (czas wykonania około 1 minuty). Wykonano 10 pomiarów dla każdej konfiguracji, a wyniki uśredniono.

Będziemy obserwować **przyspieszenie względne**, czyli odnoszące się do czasu wykonania na jednym procesorze algorytmu równoległego, a nie do czasu wykonania najlepszego algorytmu sekwencyjnego. Musimy mieć to na uwadze, ponieważ może istnieć lepszy algorytm bazowy, który nie jest obciążony m.in. kosztem komunikacji.

Wprowadzimy oznaczenia przydatne przy dalszych metrykach:

- n - liczba procesorów.
- T_1 - czas wykonania algorytmu na jednym procesorze.
- T_n - czas wykonania algorytmu na n procesorach.

Co ważne wzory na poniższe metryki różnią się w zależności od tego, czy mówimy o prawie Amdahla, czy Gustafsona.

3.2 Przyspieszenie

Dla prawa Amdahla, przyspieszenie jest definiowane jako:

$$S_n = \frac{T_1}{T(n)}$$

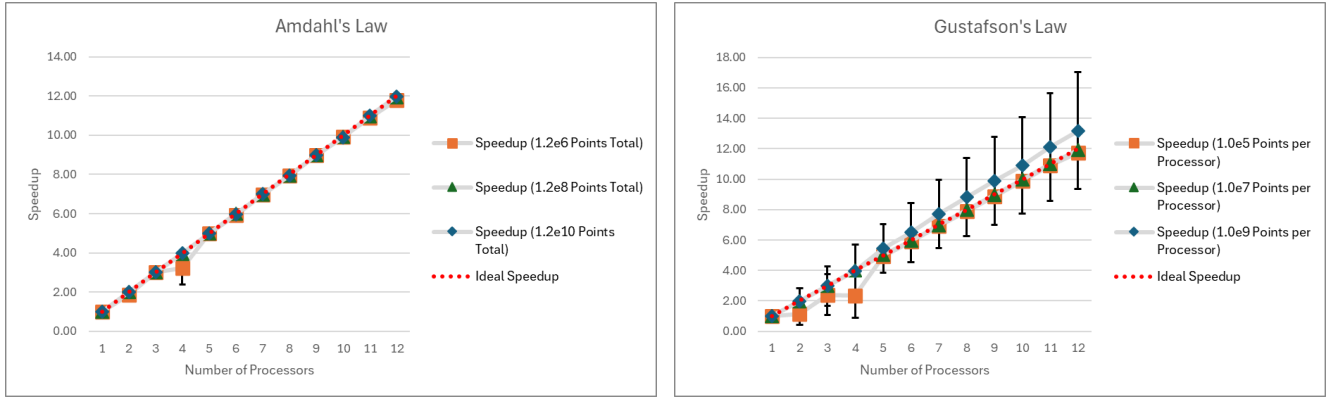
Dla prawa Gustafsona, przyspieszenie jest definiowane jako:

$$S_n = n \cdot \frac{T_1}{T(n)}$$

Odchylenie standardowe, ze względu na propagację błędów, można obliczyć jako:

$$\sigma_{S_n} = S_n \cdot \sqrt{\left(\frac{\sigma_{T_1}}{T_1}\right)^2 + \left(\frac{\sigma_{T_n}}{T_n}\right)^2}$$

Otrzymane wyniki przyspieszania przedstawiono na Rysunku 3. Są one w większości zgodne z oczekiwaniami, jednak istnieją pewne losowe odchylenia mogące wynikać z pracy komputera, na którym wykonywane były testy.



Rysunek 3: Przyspieszenie (speedup): Amdahl (lewy wykres) i Gustafson (prawy wykres), układ 1x2.

3.3 Efektywność

Dla prawa Amdahla i Gustafsona efektywność jest definiowana jako:

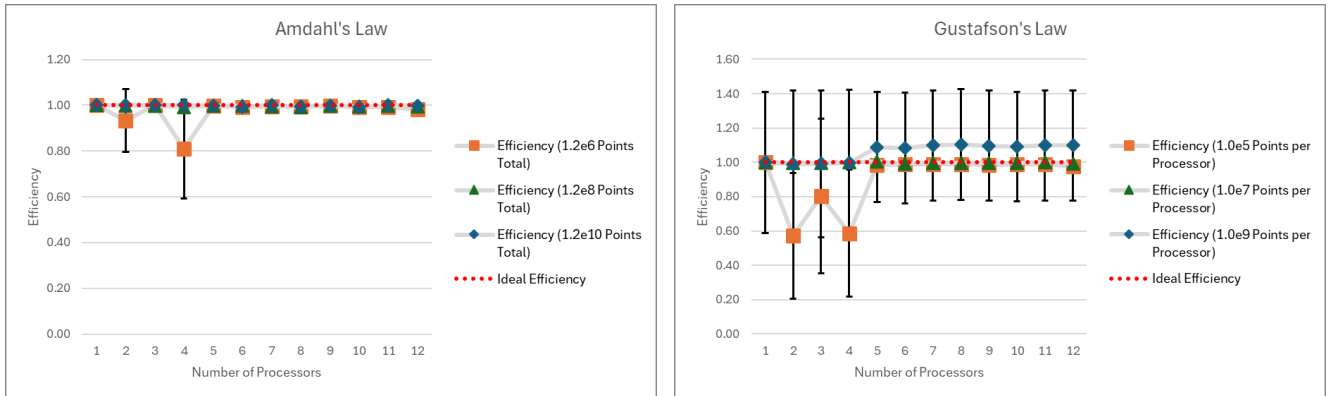
$$E_n = \frac{S_n}{n}$$

gdzie S_n jest odpowiednio przyspieszeniem dla prawa Amdahla lub Gustafsona.

Odchylenie standardowe efektywności można obliczyć jako:

$$\sigma_{E_n} = \frac{\sigma_{S_n}}{n}$$

Otrzymane wyniki efektywności przedstawiono na Rysunku 4. Są widoczne losowe odchylenia podobnie jak w powyższym przypadku dla przyspieszenia.



Rysunek 4: Efektywność (efficiency): Amdahl (lewy wykres) i Gustafson (prawy wykres), układ 1x2.

3.4 Udział części sekwencyjnej

Dla prawa Amdahla, udział części sekwencyjnej jest definiowany jako:

$$f = \frac{\frac{1}{S_n} - \frac{1}{n}}{1 - \frac{1}{n}}$$

Dla prawa Gustafsona, udział części sekwencyjnej jest definiowany jako:

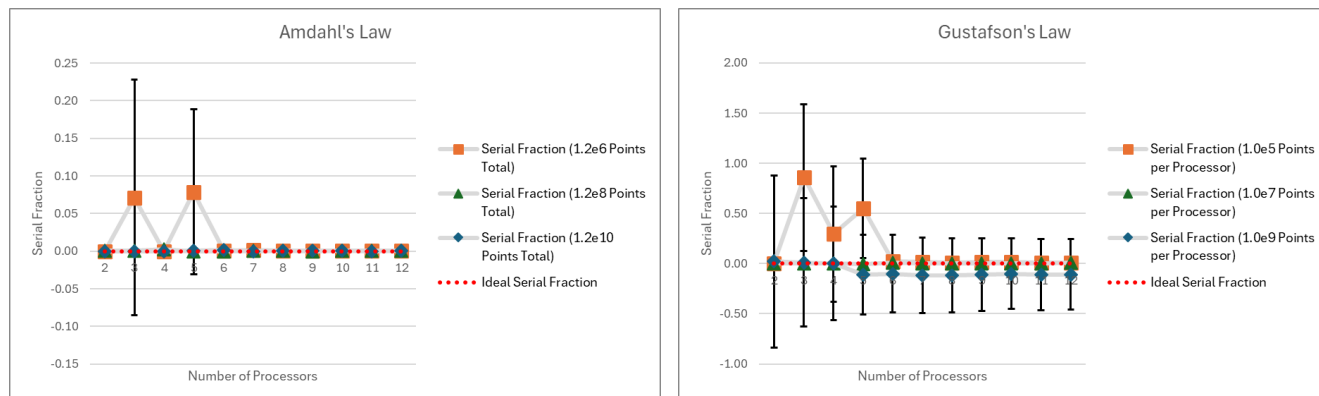
$$f = \frac{n - S_n}{n - 1}$$

gdzie S_n jest odpowiednio przyspieszeniem dla prawa Amdahla lub Gustafsona.

Odchylenie standardowe udziału części sekwencyjnej można obliczyć jako:

$$\sigma_f = \frac{\sigma_{S_n}}{n - 1}$$

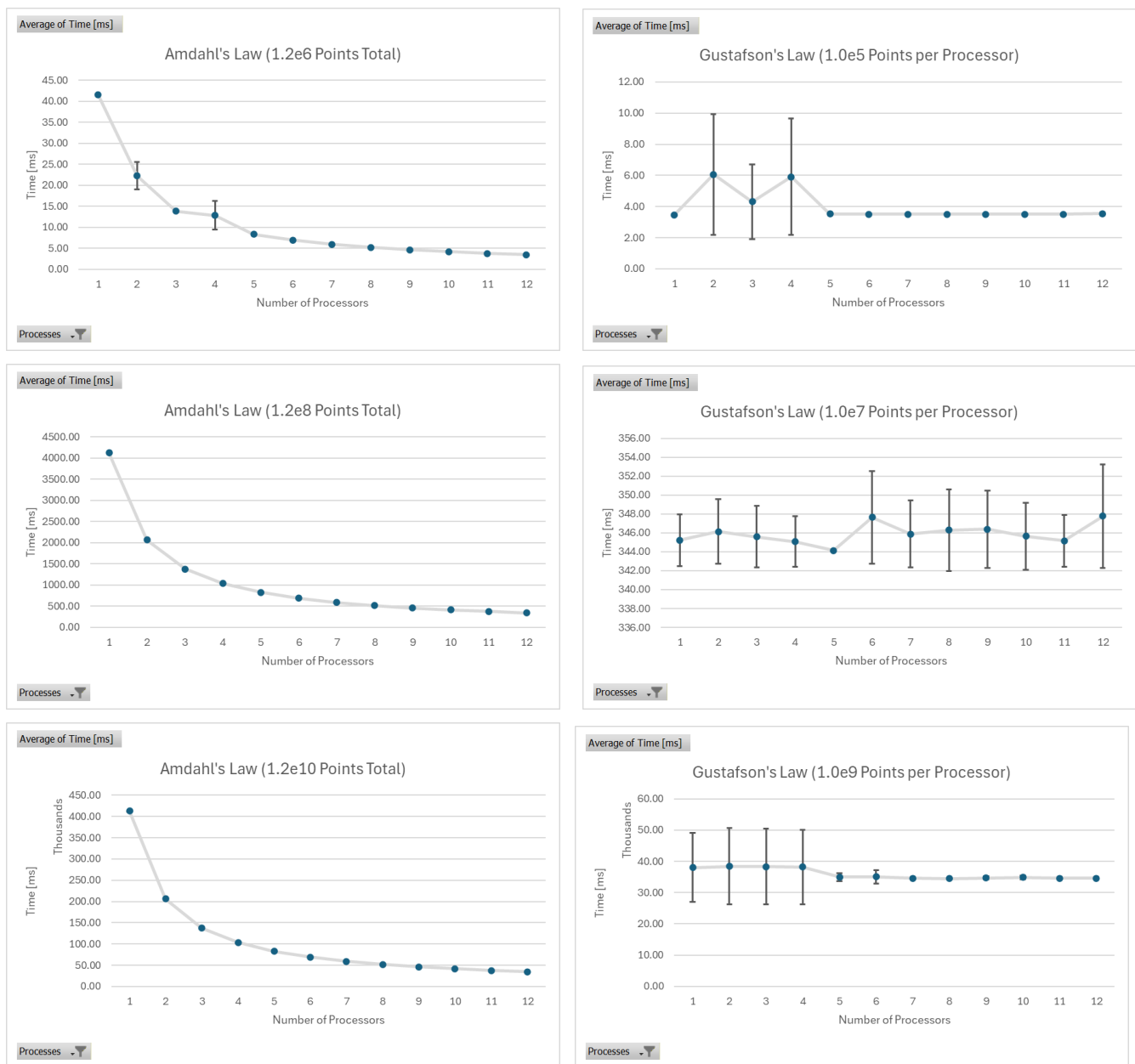
Otrzymane wyniki udziału części sekwencyjnej przedstawiono na Rysunku 5. Są one zgodne z oczekiwaniami, oraz także zawierają losowe odchylenia podobnie jak w powyższych przypadkach dla przyspieszenia i efektywności.



Rysunek 5: Udział części sekwencyjnej (serial fraction): Amdahl (lewy wykres) i Gustafson (prawy wykres), układ 1x2.

3.5 Czasy wykonania

Przedstawione wyniki czasów wykonania (Rysunek 6) pokazują, że czas wykonania maleje wraz ze wzrostem liczby procesorów, dla prawa Amdahla. Dla prawa Gustafsona, czas wykonania pozostaje mniej więcej stały, co jest zgodne z oczekiwaniami, ponieważ rozmiar problemu rośnie proporcjonalnie do liczby procesorów. Słupki błędów pokazują odchylenie standardowe, które mogą wskazywać na niestabilność pomiarów, co przełożyło się na wyniki metryk przyspieszenia, efektywności i udziału części sekwencyjnej.



Rysunek 6: Czas wykonania w zależności od liczby procesorów (układ 3x2) dla konfiguracji Amdahla i Gustafsona.

3.6 Koszt komunikacji

Narzut komunikacji jest nieznaczny, przy przesyłaniu danych typu `uint64_t` w części 1. (rysunek 1) koszt przesyłania komunikatów o rozmiarze 8B wynosił jedynie 0.0005 ms, co jest znacznie mniej niż czas wykonania algorytmu nawet dla małego rozmiaru problemu (5 ms). Mimo, że wykorzystujemy operację `MPI_Reduce`, nie powinna ona znacząco wpłynąć na wyniki.

4 Podsumowanie

Najważniejsze wnioski z przeprowadzonych testów to:

- Komunikacja wewnątrz węzła jest znacznie szybsza niż komunikacja między węzłami.
- Potrzebny jest dobór odpowiedniego rozmiaru komunikatów, aby osiągnąć optymalną przepustowość.
- Algorytm Monte Carlo do przybliżania liczby π jest naturalnie równoległym, w którym część skewencyjna i narzut komunikacyjny są pomijalnie małe.
- Każde pomiary są obarczone pewnym błędem, dlatego ważne jest wykonywanie wielu prób i analiza zgodności wyników z oczekiwaniami.